

eVenu360 Detailed Build Specifications

Complete Zone-by-Zone Technical Architecture with Temporal Sequencing

ZONE 1: PREPARE

Blueprint Creation & Compliance Baseline

Temporal Frame: T-90 to T-0 (Days Before Event)

1.1 Digital Twin Blueprint Builder

Database Schema

sql

```
SQL
-- Core Venue Structure
CREATE TABLE venues (
  venue_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  venue_name VARCHAR(255) NOT NULL,
  venue_type VARCHAR(50), -- sports, entertainment, convention,
assembly
  total_capacity INTEGER NOT NULL,
  address JSONB,
  coordinates GEOGRAPHY(POINT),
  timezone VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  blueprint_version INTEGER DEFAULT 1,
  certification_level VARCHAR(20) -- bronze, silver, gold,
platinum
```

```

);

-- Floor Plans & Spatial Data
CREATE TABLE floor_plans (
  floor_plan_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  venue_id UUID REFERENCES venues(venue_id),
  level_number INTEGER, -- -2 (parking), -1 (lower), 0 (ground),
1+ (upper)
  level_name VARCHAR(100), -- "Main Concourse", "Upper Bowl", etc.
  svg_data TEXT, -- Vector floor plan
  image_url VARCHAR(500), -- Backup raster image
  scale_factor DECIMAL(10,4), -- pixels to feet/meters
  total_area_sqft DECIMAL(10,2),
  upload_date TIMESTAMP DEFAULT NOW(),
  status VARCHAR(20) DEFAULT 'draft' -- draft, review, approved,
active
);

-- Zones (Operational Areas)
CREATE TABLE venue_zones (
  zone_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  venue_id UUID REFERENCES venues(venue_id),
  floor_plan_id UUID REFERENCES floor_plans(floor_plan_id),
  zone_name VARCHAR(255) NOT NULL,
  zone_type VARCHAR(50), -- seating, concourse, boh, parking,
medical, security
  zone_code VARCHAR(20), -- "SEC-101", "CONC-A", etc.
  capacity INTEGER,
  area_sqft DECIMAL(10,2),
  geometry GEOGRAPHY(POLYGON), -- GIS polygon for precise mapping
  svg_path TEXT, -- SVG path data for UI rendering
  accessibility_level VARCHAR(20), -- full, partial, none
  emergency_shelter BOOLEAN DEFAULT false,

  -- Operational metadata
  normal_ops_config JSONB, -- staffing, equipment, hours

```

```

    event_ops_config JSONB, -- event-specific overrides
    emergency_config JSONB, -- evacuation routes, capacity limits

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Assets (Equipment & Infrastructure)
CREATE TABLE venue_assets (
    asset_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    venue_id UUID REFERENCES venues(venue_id),
    zone_id UUID REFERENCES venue_zones(zone_id),

    -- Asset identification
    asset_tag VARCHAR(50) UNIQUE, -- Physical tag ID
    asset_name VARCHAR(255) NOT NULL,
    asset_type VARCHAR(50), -- hvac, electrical, fire, medical,
communication
    asset_category VARCHAR(50), -- AED, fire_extinguisher,
exit_sign, etc.
    manufacturer VARCHAR(100),
    model_number VARCHAR(100),
    serial_number VARCHAR(100),

    -- Location
    coordinates GEOGRAPHY(POINT),
    floor_level INTEGER,
    location_description TEXT,

    -- Status & Lifecycle
    status VARCHAR(20) DEFAULT 'active', -- active, maintenance,
retired
    install_date DATE,
    last_inspection DATE,
    next_inspection DATE,
    warranty_expiration DATE,

```

```

    expected_life_years INTEGER,

    -- Operational data
    specifications JSONB, -- capacity, coverage_radius, etc.
    maintenance_schedule JSONB,
    certifications JSONB,
    vendor_info JSONB,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Critical Infrastructure Mapping
CREATE TABLE infrastructure_systems (
    system_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    venue_id UUID REFERENCES venues(venue_id),
    system_type VARCHAR(50), -- hvac, electrical, water, fire,
communication
    system_name VARCHAR(255),

    -- System hierarchy
    parent_system_id UUID REFERENCES
infrastructure_systems(system_id),
    is_critical BOOLEAN DEFAULT false,
    is_redundant BOOLEAN DEFAULT false,

    -- Coverage
    coverage_zones UUID[], -- Array of zone_ids
    coverage_geometry GEOGRAPHY(MULTIPOLYGON),

    -- Control & Monitoring
    control_location VARCHAR(255),
    shutoff_location VARCHAR(255),
    monitoring_endpoint VARCHAR(500), -- API endpoint or IoT feed
    normal_operating_params JSONB,
    alert_thresholds JSONB,

```

```

-- Integration
vendor_system_id VARCHAR(100),
api_integration JSONB,

created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW()
);

-- Entry/Exit Points
CREATE TABLE access_points (
    access_point_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    venue_id UUID REFERENCES venues(venue_id),
    zone_id UUID REFERENCES venue_zones(zone_id),

    point_type VARCHAR(50), -- main_entry, emergency_exit,
service_entry, vip_access
    point_name VARCHAR(255),
    coordinates GEOGRAPHY(POINT),
    floor_level INTEGER,

-- Capacity & Flow
width_feet DECIMAL(5,2),
capacity_per_minute INTEGER,
egress_only BOOLEAN DEFAULT false,

-- Access Control
access_control_type VARCHAR(50), -- open, ticketed,
credentialed, restricted
access_control_system VARCHAR(100),

-- Emergency use
emergency_exit BOOLEAN DEFAULT false,
exit_discharge_location VARCHAR(255),

-- Status

```

```

    status VARCHAR(20) DEFAULT 'active', -- active, closed,
maintenance

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Emergency Routes
CREATE TABLE evacuation_routes (
    route_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    venue_id UUID REFERENCES venues(venue_id),

    route_name VARCHAR(255),
    origin_zone_id UUID REFERENCES venue_zones(zone_id),
    destination_access_point_id UUID REFERENCES
access_points(access_point_id),

    -- Route geometry
    path_geometry GEOGRAPHY(LINESTRING),
    svg_path TEXT,

    -- Route characteristics
    distance_feet DECIMAL(10,2),
    estimated_time_seconds INTEGER,
    capacity_per_minute INTEGER,
    accessible BOOLEAN DEFAULT false,

    -- Conditions
    primary_route BOOLEAN DEFAULT true,
    alternate_route_id UUID REFERENCES evacuation_routes(route_id),
    usable_conditions JSONB, -- weather, time of day, event type

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

```

API Endpoints: Blueprint Builder

javascript

JavaScript

```
// Digital Twin Blueprint API

// =====
// VENUE MANAGEMENT
// =====

/**
 * POST /api/v1/venues
 * Create new venue
 */
{
  endpoint: "POST /api/v1/venues",
  auth: "required",
  body: {
    venue_name: "Example Stadium",
    venue_type: "sports_outdoor",
    total_capacity: 65000,
    address: {
      street: "123 Stadium Way",
      city: "Atlanta",
      state: "GA",
      zip: "30303",
      country: "USA"
    },
    coordinates: {
      latitude: 33.7490,
      longitude: -84.3880
    },
    timezone: "America/New_York"
  },
  response: {
    venue_id: "uuid",
    status: "created",
    next_steps: [
```

```

        "Upload floor plans",
        "Define zones",
        "Add assets"
    ]
}
}

/**
 * GET /api/v1/venues/:venue_id
 * Retrieve venue details
 */
{
  endpoint: "GET /api/v1/venues/:venue_id",
  auth: "required",
  response: {
    venue_id: "uuid",
    venue_name: "Example Stadium",
    blueprint_completion: 67, // percentage
    compliance_score: 92,
    certification_level: "silver",
    last_updated: "2025-01-15T10:30:00Z",
    stats: {
      total_zones: 45,
      total_assets: 1247,
      floor_plans: 4,
      compliance_items: 156
    }
  }
}

// =====
// FLOOR PLAN MANAGEMENT
// =====

/**
 * POST /api/v1/venues/:venue_id/floor-plans

```

```

* Upload floor plan
*/
{
  endpoint: "POST /api/v1/venues/:venue_id/floor-plans",
  auth: "required",
  content_type: "multipart/form-data",
  body: {
    level_number: 0,
    level_name: "Main Concourse",
    file: "<binary>", // PDF, PNG, JPG, SVG, DWG
    scale_info: {
      scale_factor: 0.25, // inches per foot
      units: "feet"
    }
  },
  response: {
    floor_plan_id: "uuid",
    status: "processing",
    vectorization_job_id: "uuid",
    estimated_completion: "2 minutes"
  }
}

/**
* GET /api/v1/venues/:venue_id/floor-plans/:floor_plan_id
* Retrieve floor plan with zones
*/
{
  endpoint: "GET
/api/v1/venues/:venue_id/floor-plans/:floor_plan_id",
  auth: "required",
  query: {
    include_zones: true,
    include_assets: true,
    format: "svg" // svg, geojson, png
  },

```

```

response: {
  floor_plan_id: "uuid",
  level_name: "Main Concourse",
  svg_data: "<svg>...</svg>",
  zones: [...],
  assets: [...]
}
}

// =====
// ZONE DEFINITION
// =====

/**
 * POST /api/v1/venues/:venue_id/zones
 * Define operational zone
 */
{
  endpoint: "POST /api/v1/venues/:venue_id/zones",
  auth: "required",
  body: {
    floor_plan_id: "uuid",
    zone_name: "Section 101",
    zone_type: "seating",
    zone_code: "SEC-101",
    capacity: 850,

    // Geometry (GeoJSON or SVG path)
    geometry: {
      type: "Polygon",
      coordinates: [[[x1,y1], [x2,y2], ...]]
    },

    // OR for simple shapes
    svg_path: "M 100,100 L 200,100 L 200,200 L 100,200 Z",
  }
}

```

```

// Operational configuration
normal_ops_config: {
  staffing: {
    ushers: 2,
    security: 1
  },
  services: ["concessions", "restrooms"]
},

emergency_config: {
  evacuation_routes: ["route_id_1", "route_id_2"],
  assembly_area: "Parking Lot A",
  special_needs: false
},

accessibility_level: "full"
},
response: {
  zone_id: "uuid",
  calculated_area_sqft: 8500.5,
  capacity_per_sqft: 0.1,
  compliance_checks: {
    egress_capacity: "pass",
    accessibility: "pass",
    life_safety: "pass"
  }
}
}

/**
 * PUT /api/v1/venues/:venue_id/zones/:zone_id
 * Update zone definition
 */
{
  endpoint: "PUT /api/v1/venues/:venue_id/zones/:zone_id",
  auth: "required",

```

```
body: {
  // Any zone fields to update
  capacity: 900,
  event_ops_config: {
    concert_mode: {
      standing_room: true,
      capacity_multiplier: 1.3
    }
  }
},
response: {
  zone_id: "uuid",
  updated_fields: ["capacity", "event_ops_config"],
  compliance_recheck_required: true
}
}
```

```
// =====
// ASSET REGISTRY
// =====
```

```
/**
 * POST /api/v1/venues/:venue_id/assets
 * Register asset
 */
{
  endpoint: "POST /api/v1/venues/:venue_id/assets",
  auth: "required",
  body: {
    asset_tag: "AED-MC-001",
    asset_name: "AED Unit - Main Concourse 1",
    asset_type: "medical",
    asset_category: "AED",

    manufacturer: "Philips",
    model_number: "HeartStart FRx",
```

```
    serial_number: "12345678",

    // Location
    zone_id: "uuid",
    coordinates: {
      latitude: 33.7491,
      longitude: -84.3881,
      floor_level: 0
    },
    location_description: "Section 105 concessions, mounted on
wall near restrooms",

    // Lifecycle
    install_date: "2023-06-15",
    warranty_expiration: "2028-06-15",
    expected_life_years: 10,

    // Specifications
    specifications: {
      coverage_radius_feet: 300,
      battery_type: "Lithium",
      battery_life_years: 4,
      pads_expiration: "2026-12-31"
    },

    // Maintenance
    maintenance_schedule: {
      frequency: "monthly",
      last_check: "2025-01-10",
      next_check: "2025-02-10",
      checklist: [
        "Verify ready light",
        "Check pad expiration",
        "Inspect case condition"
      ]
    }
  },
}
```

```
    certifications: {
      fda_approved: true,
      aed_certification: "CLASS-III"
    }
  },
  response: {
    asset_id: "uuid",
    qr_code_url: "https://evenu360.com/assets/qr/uuid.png",
    nfc_tag_data: "...",
    next_inspection_alert: "2025-02-08T09:00:00Z"
  }
}

/**
 * GET /api/v1/venues/:venue_id/assets
 * Query assets
 */
{
  endpoint: "GET /api/v1/venues/:venue_id/assets",
  auth: "required",
  query: {
    asset_type: "medical", // filter
    zone_id: "uuid", // filter
    status: "active", // filter
    inspection_overdue: false, // filter

    // Spatial query
    near: {
      latitude: 33.7491,
      longitude: -84.3881,
      radius_feet: 500
    },

    // Pagination
    page: 1,
```

```

    per_page: 50,

    // Include related data
    include_maintenance_history: true
  },
  response: {
    total: 47,
    page: 1,
    per_page: 50,
    assets: [
      {
        asset_id: "uuid",
        asset_tag: "AED-MC-001",
        asset_name: "...",
        status: "active",
        distance_feet: 125.5,
        next_inspection: "2025-02-10",
        inspection_status: "current"
      },
      // ... more assets
    ]
  }
}

/**
 * POST /api/v1/venues/:venue_id/assets/:asset_id/inspect
 * Record inspection
 */
{
  endpoint: "POST
/api/v1/venues/:venue_id/assets/:asset_id/inspect",
  auth: "required",
  body: {
    inspection_date: "2025-01-30T14:30:00Z",
    inspector_id: "user_uuid",
    inspection_type: "routine", // routine, emergency, repair

```

```
    checklist_results: {
      ready_light: "pass",
      pads_condition: "pass",
      pads_expiration: "2026-12-31",
      case_condition: "pass",
      location_accessible: "pass"
    },

    issues_found: [],

    photos: [
      "url_to_photo_1",
      "url_to_photo_2"
    ],

    notes: "Unit in excellent condition. All systems
operational."
  },
  response: {
    inspection_id: "uuid",
    next_inspection_date: "2025-02-28",
    compliance_status: "current",
    certificate_url: "https://evenu360.com/certificates/uuid.pdf"
  }
}

// =====
// INFRASTRUCTURE SYSTEMS
// =====

/**
 * POST /api/v1/venues/:venue_id/systems
 * Register infrastructure system
 */
{
```

```
endpoint: "POST /api/v1/venues/:venue_id/systems",
auth: "required",
body: {
  system_type: "hvac",
  system_name: "HVAC Zone 1 - Main Bowl",
  parent_system_id: null, // top-level system
  is_critical: true,
  is_redundant: false,

  coverage_zones: ["zone_id_1", "zone_id_2", "zone_id_3"],

  control_location: "Mechanical Room B, Level -1",
  shutoff_location: "Emergency Panel MP-12",

  // IoT integration
  monitoring_endpoint:
"https://building-automation.venue.com/api/hvac/zone1",
  normal_operating_params: {
    temperature_range: [68, 74],
    humidity_range: [40, 60],
    air_changes_per_hour: 8
  },

  alert_thresholds: {
    temperature_high: 78,
    temperature_low: 65,
    humidity_high: 70,
    filter_pressure_high: 0.8
  },

  // Vendor integration
  vendor_system_id: "HVAC-CTRL-001",
  api_integration: {
    type: "bacnet",
    endpoint: "bacnet://192.168.1.50:47808",
    device_id: 100,
```

```
        credentials_vault_key: "hvac_bacnet_creds"
    },
    response: {
        system_id: "uuid",
        monitoring_active: true,
        current_status: {
            temperature: 72,
            humidity: 52,
            operational: true,
            last_check: "2025-01-30T15:45:00Z"
        },
        subsystems_detected: 3
    }
}

// =====
// ACCESS POINTS & EVACUATION ROUTES
// =====

/**
 * POST /api/v1/venues/:venue_id/access-points
 * Define entry/exit point
 */
{
    endpoint: "POST /api/v1/venues/:venue_id/access-points",
    auth: "required",
    body: {
        point_type: "main_entry",
        point_name: "Gate A",
        zone_id: "uuid",

        coordinates: {
            latitude: 33.7492,
            longitude: -84.3879,
            floor_level: 0
        }
    }
}
```

```

    },

    width_feet: 20,
    capacity_per_minute: 600,

    access_control_type: "ticketed",
    access_control_system: "Ticketmaster Access Control",

    emergency_exit: true,
    exit_discharge_location: "North Plaza"
  },
  response: {
    access_point_id: "uuid",
    calculated_egress_time: 108, // seconds to evacuate connected
    zones
    compliance_status: "pass"
  }
}

/**
 * POST /api/v1/venues/:venue_id/evacuation-routes
 * Define evacuation route
 */
{
  endpoint: "POST /api/v1/venues/:venue_id/evacuation-routes",
  auth: "required",
  body: {
    route_name: "Section 101 to Gate A",
    origin_zone_id: "uuid",
    destination_access_point_id: "uuid",

    // Path geometry (GeoJSON LineString or SVG path)
    path_geometry: {
      type: "LineString",
      coordinates: [[x1,y1], [x2,y2], [x3,y3]]
    },

```

```

        distance_feet: 285,
        capacity_per_minute: 400,
        accessible: true,
        primary_route: true
    },
    response: {
        route_id: "uuid",
        estimated_time_seconds: 142,
        alternate_route_id: "uuid", // auto-calculated alternate
        conflict_warnings: []
    }
}

```

UI Component Specifications: Blueprint Builder

typescript

```

TypeScript
// =====
// BLUEPRINT BUILDER - MAIN INTERFACE
// =====

interface BlueprintBuilderProps {
    venueId: string;
    mode: 'create' | 'edit' | 'view';
}

const BlueprintBuilder: React.FC<BlueprintBuilderProps> = ({
    venueId, mode }) => {
    return (
        <div className="blueprint-builder h-screen flex">

            {/* Left Sidebar - Tools & Layers */}
            <aside className="w-80 bg-white border-r border-gray-200">
                <BlueprintToolbar mode={mode} />

```

```

        <LayerManager venueId={venueId} />
        <AssetLibrary />
    </aside>

    {/* Center Canvas - Interactive Floor Plan */}
    <main className="flex-1 relative">
        <InteractiveCanvas venueId={venueId} mode={mode} />
        <CanvasControls />
    </main>

    {/* Right Sidebar - Properties & Compliance */}
    <aside className="w-96 bg-white border-l border-gray-200">
        <PropertiesPanel />
        <ComplianceChecker />
    </aside>

</div>
);
};

// =====
// INTERACTIVE CANVAS
// =====

interface InteractiveCanvasProps {
    venueId: string;
    mode: 'create' | 'edit' | 'view';
}

const InteractiveCanvas: React.FC<InteractiveCanvasProps> = ({
    venueId, mode }) => {
    const canvasRef = useRef<HTMLDivElement>(null);
    const [viewBox, setViewBox] = useState({ x: 0, y: 0, width:
1000, height: 1000 });
    const [selectedLayer, setSelectedLayer] =
    useState<LayerType>('floor_plan');

```

```

    const [selectedElement, setSelectedElement] = useState<string |
null>(null);

    return (
      <div ref={canvasRef} className="w-full h-full relative
bg-gray-50">

        {/* SVG Viewport */}
        <svg
          viewBox={`${viewBox.x} ${viewBox.y} ${viewBox.width}
${viewBox.height}`}
          className="w-full h-full"
          onWheel={handleZoom}
          onMouseDown={handlePanStart}
        >

          {/* Layer: Floor Plan */}
          {selectedLayer === 'floor_plan' && (
            <g id="floor-plan-layer">
              <FloorPlanLayer venueId={venueId} />
            </g>
          )}

          {/* Layer: Zones */}
          {selectedLayer === 'zones' && (
            <g id="zones-layer">
              <ZonesLayer
                venueId={venueId}
                onSelect={setSelectedElement}
                mode={mode}
              />
            </g>
          )}

          {/* Layer: Assets */}
          {selectedLayer === 'assets' && (

```

```

        <g id="assets-layer">
            <AssetsLayer
                venueId={venueId}
                onSelect={setSelectedElement}
            />
        </g>
    )}

    {/* Layer: Infrastructure */}
    {selectedLayer === 'infrastructure' && (
        <g id="infrastructure-layer">
            <InfrastructureLayer venueId={venueId} />
        </g>
    )}

    {/* Layer: Emergency */}
    {selectedLayer === 'emergency' && (
        <g id="emergency-layer">
            <EmergencyLayer venueId={venueId} />
        </g>
    )}

</svg>

{/* Overlay: Measurement Tools */}
{mode !== 'view' && <MeasurementOverlay />}

{/* Overlay: Grid */}
<GridOverlay visible={true} spacing={50} />

</div>
);
};

// =====
// ZONE DRAWING TOOL

```

```
// =====

interface ZoneDrawingToolProps {
  floorPlanId: string;
  onComplete: (zone: ZoneDefinition) => void;
}

const ZoneDrawingTool: React.FC<ZoneDrawingToolProps> = ({
  floorPlanId, onComplete }) => {
  const [points, setPoints] = useState<Point[]>([]);
  const [drawingMode, setDrawingMode] = useState<'polygon' |
'rectangle' | 'circle'>('polygon');

  const handleCanvasClick = (event: React.MouseEvent) => {
    const point = getCanvasCoordinates(event);

    if (drawingMode === 'polygon') {
      setPoints([...points, point]);
    } else if (drawingMode === 'rectangle' && points.length ===
1) {
      // Complete rectangle with second point
      const zone = createRectangleZone(points[0], point);
      onComplete(zone);
      setPoints([]);
    }
  };

  const handleDoubleClick = () => {
    if (drawingMode === 'polygon' && points.length >= 3) {
      const zone = createPolygonZone(points);
      onComplete(zone);
      setPoints([]);
    }
  };

  return (
```

```

<div className="zone-drawing-tool">

  {/* Drawing Mode Selector */}
  <div className="flex gap-2 p-4 bg-white border-b">
    <button
      className={`px-4 py-2 rounded ${drawingMode ===
'polygon' ? 'bg-teal-600 text-white' : 'bg-gray-100'}`}
      onClick={() => setDrawingMode('polygon')}
    >
      <PolygonIcon /> Polygon
    </button>
    <button
      className={`px-4 py-2 rounded ${drawingMode ===
'rectangle' ? 'bg-teal-600 text-white' : 'bg-gray-100'}`}
      onClick={() => setDrawingMode('rectangle')}
    >
      <RectangleIcon /> Rectangle
    </button>
    <button
      className={`px-4 py-2 rounded ${drawingMode ===
'circle' ? 'bg-teal-600 text-white' : 'bg-gray-100'}`}
      onClick={() => setDrawingMode('circle')}
    >
      <CircleIcon /> Circle
    </button>
  </div>

  {/* Active Drawing Preview */}
  {points.length > 0 && (
    <svg className="absolute inset-0 pointer-events-none">
      <polyline
        points={points.map(p => `${p.x},${p.y}`).join(' ')}
        stroke="#3A7B8A"
        strokeWidth="2"
        fill="rgba(58, 123, 138, 0.1)"
        strokeDasharray="5,5"
      />
    </svg>
  )}

```

```

        />
        {points.map((point, index) => (
            <circle
                key={index}
                cx={point.x}
                cy={point.y}
                r="5"
                fill="#9AAF45"
            />
        ))}
    </svg>
)}

    {/* Instructions */}
    <div className="absolute bottom-4 left-4 bg-white p-4
rounded shadow-lg">
        {drawingMode === 'polygon' && (
            <p className="text-sm text-gray-600">
                Click to add points. Double-click to complete zone.
            </p>
        )}
        {drawingMode === 'rectangle' && (
            <p className="text-sm text-gray-600">
                Click two opposite corners to create rectangle.
            </p>
        )}
    </div>

    </div>
    );
};

// =====
// ASSET PLACEMENT TOOL
// =====

```

```

interface AssetPlacementToolProps {
  assetType: string;
  onPlace: (asset: AssetPlacement) => void;
}

const AssetPlacementTool: React.FC<AssetPlacementToolProps> = ({
  assetType, onPlace }) => {
  const [ghostPosition, setGhostPosition] = useState<Point |
  null>(null);
  const [snapping, setSnapping] = useState(true);

  const handleMouseMove = (event: React.MouseEvent) => {
    let position = getCanvasCoordinates(event);

    if (snapping) {
      position = snapToGrid(position, 10); // Snap to 10px grid
    }

    setGhostPosition(position);
  };

  const handleClick = (event: React.MouseEvent) => {
    if (ghostPosition) {
      const asset: AssetPlacement = {
        asset_type: assetType,
        coordinates: ghostPosition,
        zone_id: getZoneAtPoint(ghostPosition)
      };

      onPlace(asset);
    }
  };

  return (
    <div
      className="asset-placement-tool cursor-crosshair"

```

```

      onMouseMove={handleMouseMove}
      onClick={handleClick}
    >

    {/* Ghost Asset Preview */}
    {ghostPosition && (
      <AssetIcon
        type={assetType}
        x={ghostPosition.x}
        y={ghostPosition.y}
        opacity={0.5}
      />
    )}

    {/* Snapping Toggle */}
    <div className="absolute top-4 right-4">
      <label className="flex items-center gap-2 bg-white p-2
rounded shadow">
        <input
          type="checkbox"
          checked={snapping}
          onChange={(e) => setSnapping(e.target.checked)}
        />
        <span className="text-sm">Snap to Grid</span>
      </label>
    </div>

  </div>
);
};

```

Temporal Sequence: PREPARE Zone

javascript

JavaScript

```
// =====  
// PREPARE ZONE TEMPORAL SEQUENCING  
// T-90 to T-0 (Days before event)  
// =====  
  
const PREPARE_SEQUENCE = {  
  
  // =====  
  // T-90 to T-60: INITIAL BLUEPRINT CREATION  
  // =====  
  phase_1_initial_setup: {  
    timeline: "T-90 to T-60",  
    duration_days: 30,  
  
    milestones: [  
      {  
        day: "T-90",  
        task: "Venue Registration",  
        required_data: [  
          "Venue name and type",  
          "Total capacity",  
          "Physical address",  
          "Operating hours",  
          "Contact information"  
        ],  
        system_actions: [  
          "Create venue record",  
          "Generate venue_id",  
          "Initialize compliance matrix",  
          "Setup user permissions"  
        ],  
        deliverable: "Venue profile created"  
      },  
  
      {  
        day: "T-87",
```

```
    task: "Floor Plan Upload",
    required_data: [
      "PDF/CAD floor plans",
      "Scale information",
      "Level designations",
      "North orientation"
    ],
    system_actions: [
      "Process uploaded files",
      "Vectorize raster images",
      "Extract dimensions",
      "Generate SVG layers"
    ],
    deliverable: "Floor plans digitized"
  },

  {
    day: "T-85",
    task: "Zone Definition Begin",
    required_data: [
      "Zone names and codes",
      "Zone types (seating, concourse, etc.)",
      "Capacity by zone",
      "Access control requirements"
    ],
    system_actions: [
      "Create zone boundaries",
      "Calculate areas",
      "Validate capacity ratios",
      "Map access points"
    ],
    deliverable: "Initial zones defined (target 50%)"
  },

  {
    day: "T-75",
```

```

    task: "Asset Registry Start",
    required_data: [
        "Critical asset inventory",
        "Equipment locations",
        "Inspection schedules",
        "Vendor information"
    ],
    system_actions: [
        "Create asset records",
        "Generate asset tags/QR codes",
        "Map to zones",
        "Initialize inspection schedules"
    ],
    deliverable: "Critical assets registered (AEDs, fire,
exits)"
  },

  {
    day: "T-60",
    task: "Phase 1 Review",
    validation_criteria: {
      floor_plans_complete: ">90%",
      zones_defined: ">50%",
      critical_assets_registered: ">80%",
      data_quality_score: ">85%"
    },
    system_actions: [
      "Run validation checks",
      "Generate gap report",
      "Create action items",
      "Notify stakeholders"
    ],
    deliverable: "Phase 1 completion report"
  }
]
},

```

```

// =====
// T-60 to T-30: DETAILED MODELING
// =====
phase_2_detailed_modeling: {
  timeline: "T-60 to T-30",
  duration_days: 30,

  milestones: [
    {
      day: "T-60",
      task: "Complete Zone Definition",
      target: "100% zones defined and validated",
      system_actions: [
        "Finish remaining zones",
        "Validate all boundaries",
        "Check capacity calculations",
        "Map emergency routes"
      ]
    },

    {
      day: "T-55",
      task: "Infrastructure System Mapping",
      required_data: [
        "HVAC zone coverage",
        "Electrical panel locations",
        "Water system layout",
        "Fire suppression zones",
        "Communication infrastructure"
      ],
      system_actions: [
        "Create system records",
        "Map coverage areas",
        "Define control points",
        "Setup monitoring endpoints"
      ]
    }
  ]
}

```

```
    ],  
    deliverable: "All critical systems mapped"  
  },  
  
  {  
    day: "T-50",  
    task: "Emergency Route Planning",  
    required_data: [  
      "All entry/exit points",  
      "Evacuation assembly areas",  
      "Route distances and times",  
      "Accessibility requirements"  
    ],  
    system_actions: [  
      "Calculate egress capacity",  
      "Validate route redundancy",  
      "Check code compliance",  
      "Generate route maps"  
    ],  
    deliverable: "Complete evacuation plan"  
  },  
  
  {  
    day: "T-45",  
    task: "Complete Asset Registry",  
    target: "100% assets registered",  
    system_actions: [  
      "Add all remaining assets",  
      "Verify locations",  
      "Schedule inspections",  
      "Generate maintenance calendar"  
    ]  
  },  
  
  {  
    day: "T-40",
```

```

    task: "API Integration Configuration",
    required_data: [
        "Building automation credentials",
        "Access control API keys",
        "Weather service endpoints",
        "Emergency notification systems"
    ],
    system_actions: [
        "Test all API connections",
        "Validate data flows",
        "Configure alert thresholds",
        "Setup monitoring dashboards"
    ],
    deliverable: "All integrations live"
},

{
    day: "T-30",
    task: "Phase 2 Review",
    validation_criteria: {
        zones_complete: "100%",
        assets_complete: "100%",
        systems_mapped: "100%",
        routes_validated: "100%",
        integrations_live: ">90%"
    },
    deliverable: "Complete digital twin baseline"
}
]
},

// =====
// T-30 to T-7: COMPLIANCE & VALIDATION
// =====
phase_3_compliance_validation: {
    timeline: "T-30 to T-7",

```

```
duration_days: 23,

milestones: [
  {
    day: "T-30",
    task: "Compliance Requirements Mapping",
    system_actions: [
      "Load applicable codes (building, fire, ADA)",
      "Map requirements to venue",
      "Generate compliance checklist",
      "Identify documentation needs"
    ],
    deliverable: "Complete compliance matrix"
  },

  {
    day: "T-25",
    task: "Gap Analysis",
    system_actions: [
      "Compare current state to requirements",
      "Identify non-compliance items",
      "Calculate compliance score",
      "Prioritize remediation"
    ],
    deliverable: "Gap analysis report with action plan"
  },

  {
    day: "T-20",
    task: "Evidence Collection",
    required_data: [
      "Inspection reports",
      "Certifications",
      "Training records",
      "Equipment certificates"
    ],
  },
```

```
    system_actions: [
      "Upload all evidence",
      "Link to compliance items",
      "Verify completeness",
      "Store in evidence ledger"
    ],
    deliverable: "Complete evidence package"
  },

  {
    day: "T-14",
    task: "Compliance Validation",
    system_actions: [
      "Run automated compliance checks",
      "Validate all calculations",
      "Check documentation completeness",
      "Generate compliance report"
    ],
    deliverable: "Compliance certification ready"
  },

  {
    day: "T-7",
    task: "Final Baseline Lock",
    validation_criteria: {
      blueprint_complete: "100%",
      compliance_score: ">95%",
      evidence_complete: "100%",
      integrations_tested: "100%"
    },
    system_actions: [
      "Lock baseline version",
      "Generate final reports",
      "Transition to READY zone",
      "Notify all stakeholders"
    ],
  },
```

```

        deliverable: "PREPARE zone complete - READY to proceed"
    }
]
}
};

// =====
// PREPARE ZONE STATE MACHINE
// =====

const PREPARE_STATE_MACHINE = {
  states: {
    NOT_STARTED: {
      allowed_transitions: ["INITIAL_SETUP"],
      required_data: ["venue_registration"]
    },

    INITIAL_SETUP: {
      completion_criteria: {
        floor_plans_uploaded: true,
        zones_defined_percent: 50,
        critical_assets_registered: 80
      },
      allowed_transitions: ["DETAILED_MODELING"],
      estimated_duration_days: 30
    },

    DETAILED_MODELING: {
      completion_criteria: {
        zones_complete: 100,
        assets_complete: 100,
        systems_mapped: 100,
        routes_validated: true
      },
      allowed_transitions: ["COMPLIANCE_VALIDATION"],
      estimated_duration_days: 30
    }
  }
};

```

```
    },

    COMPLIANCE_VALIDATION: {
      completion_criteria: {
        compliance_score: 95,
        evidence_complete: true,
        gaps_addressed: true
      },
      allowed_transitions: ["BASELINE_LOCKED"],
      estimated_duration_days: 23
    },

    BASELINE_LOCKED: {
      description: "PREPARE complete, ready for event
operations",
      allowed_transitions: ["READY_ZONE"],
      final_deliverables: [
        "Complete digital twin",
        "Compliance certification",
        "Evidence ledger baseline",
        "Operational playbooks"
      ]
    }
  }
};
```